

PSE-EVAL-MATRIX-01

Empirical Benchmark Matrix for Post-Symbolic Cognition Systems

Formale Implementierungsspezifikation v0.1

Sebastian Klemm

Technische Ausarbeitung fuer Coding-Agent-Implementierung

Mai 2026

Zusammenfassung

Diese Spezifikation definiert einen strukturellen Benchmark- und Evaluationsrahmen fuer post-symbolische Kognitionssysteme. Ziel ist nicht ein einfacher Benchmark-Harness, sondern eine empirische Matrix, die Systemvarianten, Workload-Familien, Domänen, Metriken, Kalibrierungszustände, Ablationen, Sicherheitsgates, Replay-Verifikation, statistische Auswertung und qualitative Evidenz zu einem reproduzierbaren Forschungsinstrument verbindet. Der Rahmen ist auf PSE, PSE-Traversal, Horizon, Cognition und verwandte post-symbolische Kontrollarchitekturen zugeschnitten, aber so formuliert, dass er als generisches Evaluationssubstrat fuer deterministische Kognitions-, Agenten- und Projektionssysteme implementiert werden kann.

Inhaltsverzeichnis

1	Status, Ziel und Geltungsbereich	5
1.1	Status	5
1.2	Ziel	5
1.3	Nicht-Zweck	5
2	Grundprinzip	5
2.1	Die Evaluationsmatrix	5
2.2	Blueprint statt Harness	6
3	Systemvarianten	6
3.1	Variant Ladder	6
3.2	Ablationspflicht	7

4	Workload-Familien	7
4.1	Pflichtfamilien	7
4.2	Domänenadapter	8
5	Metrikfamilien	8
5.1	Primäre Outcome-Metriken	8
5.2	Post-symbolische Strukturmetriken	9
5.3	Systemmetriken	9
6	Scoring-Modell	9
6.1	Keine Einzahl ohne Profil	9
6.2	Capability Profile	9
6.3	Safety-Adjusted Utility	10
6.4	Cognition Uplift	10
6.5	Layer Marginal Utility	10
7	Experimentdesign	10
7.1	Experimentarten	10
7.2	Statistische Regeln	11
8	Ground Truth und Gold Labels	11
8.1	GroundTruthProfile	11
8.2	Labelquellen	11
8.3	Gold-Label-Integrität	11
9	Kalibrierungsprotokoll	12
9.1	Train/Calibration/Test-Trennung	12
9.2	Adaptive Calibration	12
9.3	Kalibrierungsverbot im Testsplit	12
10	Datenmodelle	12
10.1	EvaluationSpec	12
10.2	SystemVariantSpec	13
10.3	WorkloadSpec	13
10.4	MetricSpec	13
10.5	RunLedger	13
10.6	TrialReport	14
10.7	MetricObservation	14

11 Cognition-spezifische Evaluierung	14
11.1 Panorama-Metriken	14
11.2 SpiralMemory-Metriken	15
11.3 Constraint-Collapse-Metriken	15
11.4 SelfModel-Metriken	15
12 Agenten- und Code-Benchmarks	15
12.1 Taskklassen	15
12.2 Code-Agent-Metriken	16
13 Failure Taxonomie	16
14 Reports	16
14.1 EvaluationSummaryReport	16
14.2 CapabilityProfile	17
14.3 AblationSummary	17
15 CLI-Schnittstelle	17
15.1 Subcommands	17
15.2 CLI-Anforderungen	18
16 Feature Flags und Modulzuschnitt	18
16.1 Rust-Modulstruktur	18
16.2 Feature Flags	18
17 Determinismus und Audit	19
17.1 Kanonisierung	19
17.2 Audit-Artifacts	19
18 Benchmark-Matrix-Presets	19
18.1 Preset: Agent Cognition	19
18.2 Preset: Streaming Event Detection	20
18.3 Preset: Post-Symbolic Ablation	20
19 Qualitative Evaluation	20
19.1 Warum qualitative Evaluation erforderlich ist	20
19.2 Rubrik	20
19.3 Review-Protokoll	20
20 Definition of Done	21

21 Coding-Agent-Auftrag	21
22 Referenz-Zustandsmaschine	22
23 Schlussformel	22

1 Status, Ziel und Geltungsbereich

1.1 Status

Dieses Dokument ist eine normative Spezifikation. Es beschreibt Datenmodelle, Experimenttypen, Metriken, Auswertungsregeln, CLI-Oberflächen, Reports, Akzeptanztests und einen Coding-Agent-Auftrag fuer eine implementierbare Evaluationsmatrix.

1.2 Ziel

PSE-EVAL-MATRIX-01 soll beweisbar machen, ob ein post-symbolisches Kognitionssystem gegen Baselines und interne Ablationen messbare Vorteile erzeugt. Der Rahmen muss insbesondere beantworten:

- erzeugt das System bessere Ergebnisse als ein naiver Agent oder ein klassischer Detektor?
- welche Schicht liefert welchen inkrementellen Nutzen?
- sind Crystals, CandidateBundles, FinalizedEmissions oder CognitionReports mit Ground Truth korreliert?
- bleibt Replay byte-identisch?
- verbessert Cognition die Auswahl der Kandidaten oder nur die interne Diagnostik?
- sind Gates zu streng, zu permissiv oder kalibrierbar?
- wie verhalten sich Durchsatz, Latenz, Speicher, Failure-Rate und Recovery?

1.3 Nicht-Zweck

Der Rahmen ist nicht:

- ein einzelnes Benchmark-Skript;
- eine Marketing-Demo;
- ein Ersatz für Domänenkalibrierung;
- ein probabilistisches Evaluationstool ohne Replay;
- eine Sammlung freier Metriken ohne vorab deklarierte Auswertungsregeln.

2 Grundprinzip

2.1 Die Evaluationsmatrix

Eine Evaluation ist ein Tensor ueber Varianten, Workloads, Domänen, Kalibrierungszustände und Metriken:

$$\mathcal{E} = \mathcal{V} \times \mathcal{W} \times \mathcal{D} \times \mathcal{M} \times \mathcal{R}.$$

Eine einzelne Messung ist:

$$y_{v,w,d,m,r} = M_m(S_v, W_w, D_d, R_r),$$

wobei:

- S_v eine Systemvariante ist,
- W_w eine Workload-Familie ist,
- D_d eine Domäne oder ein Adapterprofil ist,
- M_m eine deklarierte Metrik ist,
- R_r ein deterministischer RunDescriptor ist.

Kernforderung

Jede Evaluation **MUST** so definiert sein, dass ein unabhängiger Runner aus EvaluationSpec, DatasetManifest, VariantSpec und RunDescriptor dieselben Reports, Hashes und statistischen Summaries reproduzieren kann.

2.2 Blueprint statt Harness

Ein Benchmark-Harness fuehrt Tests aus. Die Evaluationsmatrix legt fest:

- welche Fragestellung getestet wird;
- welche Systemvarianten verglichen werden;
- welche Workloads und Domänen zulaessig sind;
- welche Metriken primaer, sekundär oder diagnostisch sind;
- welche Ablationen zwingend sind;
- welche Gate- und Replay-Verletzungen ein Ergebnis ungültig machen;
- wie statistisch aggregiert wird;
- welche Evidenz fuer Publikation oder Produktentscheidung ausreicht.

3 Systemvarianten

3.1 Variant Ladder

Die Evaluationsmatrix **MUST** eine Variant Ladder unterstuetzen. Mindestens folgende Varianten **SHOULD** definierbar sein:

Variante	Bedeutung
B0_Baseline	Naiver Baseline-Agent, klassischer Detektor oder einfacher Tool-Loop ohne PSE.
B1_PSE_Core	Nur Streaming Engine, GateSnapshot, Crystal/Falsifier/Replay.
B2_Traversal	ProblemSpec, FieldCube, DoFGraph, CollapsePlan, Candidate, GateReport, PSE-Bridge.
B3_Signature	Traversal plus StructuralOperator, Signature, SignatureDiagnostics, SignatureGate.

Variante	Bedeutung
B4_Dynamics	B3 plus DynamicTick/DynamicRun, GuidanceField, MorphodynamicCompressor.
B5_Horizon	B4 plus NullCenter, HorizonChart, PhaseRays, EventHorizonWindow, HorizonCrossingGate.
B6_Cognition	B5 plus CanonicalCognitionState, CognitiveState5D, SpiralMemory, ConstraintLattice, HypercubePuzzle, PhasePanorama, SelfModelTensor, HandoffGate.
B7_FullStack	Vollständige konfigurierte Schichtkette mit realer Domänenkalibrierung.

3.2 Ablationspflicht

Jede Forschungs- oder Produktbehauptung **MUST** mindestens eine Ablation enthalten:

- ohne Cognition;
- ohne SpiralMemory;
- ohne ConstraintLattice;
- ohne PhasePanorama;
- ohne GovernedWormholes;
- ohne SelfModelTensor;
- ohne Adaptive Calibration;
- Gate diagnostic-only vs. fail-closed.

4 Workload-Familien

4.1 Pflichtfamilien

Die Matrix **MUST** mehrere Workload-Familien abdecken. Eine einzelne erfolgreiche Demo ist nicht hinreichend.

Familie	Zweck	Primaere Metriken
StreamEvent	strukturelle Ereignisse in Beobachtungsstreams erkennen.	Precision, Recall, F1, FalseCrystalRate, latency.
AnomalyRegime	Regimewechsel, Drift, Zustandsbrueche.	detection delay, false alarms, stability.
TraversalPuzzle	strukturierte Problemräume kollabieren.	solved rate, entropy reduction, collapse correctness.
CodeAgentPatch	Coding-Agent-Aufgaben mit Tests und Repo-Kontext.	test pass rate, regression rate, iterations, manual repair.

Familie	Zweck	Primaere Metriken
DocSynthesis	PDF-/Dokumentenrohstoff zu Spezifikationen extrahieren.	extraction fidelity, primitive utility, contradiction handling.
MemoryReuse	fruehere Loesungsbahnen wiederfinden.	hit utility, retrieval precision, reuse success.
HorizonFinalization	Kandidaten korrekt finalisieren oder halten.	finalize precision, hold precision, replay validity.
CognitionPanorama	sichtbare/latente/blockierte Pfade kartieren.	panorama coverage, path ranking quality, attractor hit rate.
MultiAgent	koordinierte Agentenpfade, Wormholes, Budgets.	coordination gain, wormhole utility, budget violations.

4.2 Domänenadapter

Die Matrix **MAY** bestehende Adapterprofile verwenden, beispielsweise Märkte, Vitaldaten, Seismologie, Wetter, Energiegrid, IoT, Syslog, Tabular oder Model Monitoring. Jede Domäne **MUST** ein DatasetManifest mit Ground Truth oder Gold Labels definieren, sofern Task-Erfolg behauptet wird.

5 Metrikfamilien

5.1 Primäre Outcome-Metriken

Metrik	Definition	Verwendung
TaskSuccess	Anteil erfolgreich abgeschlossener Aufgaben.	Hauptmetrik fuer Agenten- und Puzzleaufgaben.
PrecisionRecallF1	Standardklassifikation fuer Events/Crystals.	Event- und Anomaliedetektion.
FalseCommitRate	falsche Crystals, Emissions oder Handoffs pro Run.	Sicherheitskritische Hauptmetrik.
HoldCorrectness	Anteil korrekter Hold/Wait/Abort-Entscheidungen.	Fail-closed-Qualitaet.
ReplayIdentity	Byte-identische Wiederholung aller Reports.	Determinismuspflicht.
RegressionRate	Anteil neuer Fehler nach Patch/Action.	Coding-Agent-Aufgaben.
ManualIntervention	Anzahl menschlicher Eingriffe pro erfolgreichem Run.	Produkt- und Autonomiemetrik.

5.2 Post-symbolische Strukturmetriken

Metrik	Bedeutung
ConstraintMass	gewichtete Constraint-Masse im Lattice.
EntropyReduction	relative Reduktion des Lösungsraums.
FeasibleSetUniqueness	Eindeutigkeit der verbleibenden Kandidatenregion.
CollapseValidity	Korrektheit des EntropyCollapseCertificate.
SignatureGap	stabilisierte strukturelle Trennung in der Signature-Schicht.
DynamicEnergyDelta	Energie-/Spannungsdifferenz ueber DynamicTick.
HorizonAlignment	Phasen-, Epoch- und Sichtbarkeitsguete im Horizon-Gate.
PanoramaCoverage	Anteil relevanter sichtbarer, latenter und blockierter Pfade.
AttractorSeparation	Abstand und Rangordnung konkurrierender Attraktoren.
MemoryHitUtility	Beitrag von SpiralMemory-Hits zum Task-Erfolg.
WormholeUtility	Erfolgsgewinn kontrollierter Shortcuts minus Budgetkosten.
SelfModelDrift	Drift des Selbstmodells relativ zu StabilityBound.

5.3 Systemmetriken

- **ThroughputObsPerSec**: Beobachtungen pro Sekunde.
- **PipelineLatencyUs**: End-to-end-Latenz pro macro step oder cognition run.
- **MemoryBytes**: Speicherverbrauch pro Run.
- **ReportSizeBytes**: Audit- und Reportgroesse.
- **GateCost**: CPU-Zeit je Gate-Familie.
- **CalibrationCost**: Kosten bis stabile Kalibrierung.

6 Scoring-Modell

6.1 Keine Einzahl ohne Profil

Ein einzelner Score **MUST NOT** als alleinige Systemaussage verwendet werden. Jeder Summary-Score **MUST** gemeinsam mit einem Capability Profile ausgegeben werden.

6.2 Capability Profile

Ein Capability Profile ist ein Vektor:

$$P(S) = (U_{task}, U_{replay}, U_{safety}, U_{cognition}, U_{efficiency}, U_{calibration}, U_{robustness}).$$

- U_{task} : Task-Erfolg und Qualitaet.
- U_{replay} : Determinismus und Zertifizierbarkeit.
- U_{safety} : false commit, gate correctness, hold correctness.

- $U_{cognition}$: Panorama, Memory, Attractor, SelfModel.
- $U_{efficiency}$: Durchsatz, Latenz, Speicher.
- $U_{calibration}$: Schwellenstabilitaet und Domänenanpassung.
- $U_{robustness}$: Drift, Perturbation, Distribution Shift.

6.3 Safety-Adjusted Utility

$$SAU = U_{task} \cdot U_{replay} \cdot U_{safety} \cdot (1 - P_{catastrophic}).$$

$P_{catastrophic}$ ist kein probabilistischer Claim ohne Evidenz, sondern ein empirischer Anteil schwerer Failures innerhalb der Benchmarkmatrix.

6.4 Cognition Uplift

$$CU = Score(B6) - Score(B5)$$

Der Cognition Uplift misst den inkrementellen Beitrag der Cognition-Schicht gegen Horizon/Projection ohne Cognition.

6.5 Layer Marginal Utility

$$LMU_i = Score(B_i) - Score(B_{i-1}).$$

Eine Schicht gilt empirisch nuetzlich, wenn LMU_i positiv ist und keine sicherheitskritische Metrik signifikant schlechter wird.

7 Experimentdesign

7.1 Experimentarten

Experiment	Zweck
BaselineComparison	Vergleich gegen klassische Detektoren, naive Agenten oder einfache LLM-Tool-Loops.
AblationLadder	Isoliert den Beitrag jeder PSE-Schicht.
CalibrationCurve	Prueft Schwellen, AdaptiveCalibrator, Gate-ROC und Hold-Korrektheit.
ReplayAudit	Beweist byte-identische Wiederholung und Hash-Integritaet.
StressPerturbation	Testet Robustheit gegen Rauschen, Drift, missing data, adversarial perturbations.
LongRunStability	Misst Drift, Memory-Nutzen, CarrierMigration und SelfModel ueber lange Runs.
CrossDomainTransfer	Prueft, ob Kalibrierung und Primitive zwischen Domänen uebertragbar sind.

Experiment	Zweck
HumanReviewStudy	Erfasst manuelle Nacharbeit, Erklärbarkeit und Diagnosequalitaet.

7.2 Statistische Regeln

1. Jede Evaluation **MUST** vorab primaere und sekundäre Metriken deklarieren.
2. Jede Vergleichsaussage **MUST** ein paired design verwenden, wenn Varianten dieselben Inputs verarbeiten.
3. Confidence Intervals **SHOULD** via Bootstrap oder exakte binomiale Intervalle berechnet werden.
4. Multiple comparisons **SHOULD** markiert oder korrigiert werden.
5. Runs mit Replay-Verletzung **MUST** separat als invalid berichtet werden.
6. Missing Labels **MUST** als eigene Kategorie gefuehrt werden, nicht stillschweigend verworfen werden.

8 Ground Truth und Gold Labels

8.1 GroundTruthProfile

```
pub struct GroundTruthProfile {
  pub profile_id: Hash256,
  pub label_space: LabelSpace,
  pub label_source: LabelSource,
  pub adjudication_policy: AdjudicationPolicy,
  pub uncertainty_policy: UncertaintyPolicy,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

8.2 Labelquellen

- SyntheticExact: exakt bekannte Events, Puzzles oder Constraints.
- SemiSyntheticInjection: reale Daten plus injizierte bekannte Events.
- HistoricalOutcome: reale Outcomes mit Zeitstempel und Provenance.
- UnitTestOracle: Softwareaufgaben mit Test-Suite als Oracle.
- HumanAdjudicated: menschliche Bewertung mit Doppelannotation und Konfliktregeln.

8.3 Gold-Label-Integritaet

Gold Labels **MUST** selbst content-addressed sein. Jede Veränderung am Label-Set **MUST** die Dataset-ID ändern.

9 Kalibrierungsprotokoll

9.1 Train/Calibration/Test-Trennung

Jedes Dataset **MUST** drei logische Splits besitzen:

- **calibration**: Schwellen, Adapter und Gate-Profile kalibrieren;
- **validation**: Konfiguration auswählen;
- **test**: finalen Report erzeugen, keine Nachkalibrierung erlaubt.

9.2 Adaptive Calibration

Wenn adaptive Kalibrierung aktiv ist, **MUST** jeder Schwellenwechsel in einem Calibration-Ledger protokolliert werden.

```
pub struct CalibrationLedgerEntry {  
    pub entry_id: Hash256,  
    pub previous_threshold_hash: Hash256,  
    pub new_threshold_hash: Hash256,  
    pub evidence_window_hash: Hash256,  
    pub reason: CalibrationReason,  
    pub accepted: bool,  
}
```

9.3 Kalibrierungsverbot im Testsplit

Im finalen Testsplit **MUST** Kalibrierung eingefroren sein, ausser das Experiment ist explizit als OnlineCalibrationExperiment deklariert.

10 Datenmodelle

10.1 EvaluationSpec

```
pub struct EvaluationSpec {  
    pub spec_id: Hash256,  
    pub schema_version: String,  
    pub purpose: EvaluationPurpose,  
    pub variants: Vec<SystemVariantSpec>,  
    pub workloads: Vec<WorkloadSpec>,  
    pub datasets: Vec<DatasetManifest>,  
    pub metrics: Vec<MetricSpec>,  
    pub experiment_design: ExperimentDesign,  
    pub statistical_plan: StatisticalPlan,  
    pub determinism_policy: DeterminismPolicy,  
    pub output_policy: OutputPolicy,  
}
```

10.2 SystemVariantSpec

```
pub struct SystemVariantSpec {  
    pub variant_id: String,  
    pub layer_mask: LayerMask,  
    pub feature_flags: BTreeMap<String, bool>,  
    pub config_hash: Hash256,  
    pub rd_hash: Hash256,  
    pub solver_profile: SolverProfile,  
    pub calibration_profile: CalibrationProfile,  
}
```

10.3 WorkloadSpec

```
pub struct WorkloadSpec {  
    pub workload_id: String,  
    pub family: WorkloadFamily,  
    pub input_manifest_hash: Hash256,  
    pub ground_truth_profile: Option<Hash256>,  
    pub task_budget: TaskBudget,  
    pub success_criteria: Vec<SuccessCriterion>,  
}
```

10.4 MetricSpec

```
pub struct MetricSpec {  
    pub metric_id: String,  
    pub family: MetricFamily,  
    pub direction: MetricDirection,  
    pub primary: bool,  
    pub aggregation: AggregationPolicy,  
    pub invalidation_rules: Vec<InvalidationRule>,  
}
```

10.5 RunLedger

```
pub struct EvaluationRunLedger {  
    pub ledger_id: Hash256,  
    pub evaluation_spec_hash: Hash256,  
    pub entries: Vec<EvaluationRunEntry>,  
    pub chain_hash: Hash256,  
}  
  
pub struct EvaluationRunEntry {  
    pub entry_id: Hash256,  
    pub variant_id: String,  
    pub workload_id: String,
```

```

pub dataset_id: Hash256,
pub run_descriptor_hash: Hash256,
pub report_hash: Hash256,
pub replay_hash: Hash256,
pub status: RunStatus,
}

```

10.6 TrialReport

```

pub struct TrialReport {
  pub trial_id: Hash256,
  pub variant_id: String,
  pub workload_id: String,
  pub dataset_id: Hash256,
  pub run_descriptor_hash: Hash256,
  pub outputs: TrialOutputs,
  pub metrics: Vec<MetricObservation>,
  pub gates: GateObservationSet,
  pub replay: ReplayObservation,
  pub diagnostics: Vec<DiagnosticRecord>,
}

```

10.7 MetricObservation

```

pub struct MetricObservation {
  pub metric_id: String,
  pub value: CanonicalNumber,
  pub unit: Option<String>,
  pub valid: bool,
  pub invalid_reason: Option<String>,
  pub evidence_refs: Vec<EvidenceRef>,
}

```

11 Cognition-spezifische Evaluierung

11.1 Panorama-Metriken

Metrik	Definition
CurrentHorizonCoverage	Anteil relevanter erreichbarer Pfade im aktuellen Horizont.
CounterfactualCoverage	Anteil relevanter Alternativpfade im kontrafaktischen Horizont.
BlockedPathCorrectness	Anteil korrekt blockierter Pfade.

Metrik	Definition
AttractorRankingAccuracy	Rangkorrelation zwischen vorhergesagten Attraktoren und Task-Erfolg.
PanoramaCompressionRatio	relevante Pfadinformation pro Report-Byte.

11.2 SpiralMemory-Metriken

- HitPrecision: Anteil Memory Hits, die tatsaechlich nuetzlich waren.
- HitRecall: Anteil relevanter bekannter Loesungsbahnen, die gefunden wurden.
- ReuseGain: Task-Erfolg mit Memory minus Task-Erfolg ohne Memory.
- FalseResonanceRate: Resonanznahe, aber schaedliche Hits.

11.3 Constraint-Collapse-Metriken

- PercolationAccuracy: stimmt percolation pass mit spaeterer Loesbarkeit ueberein?
- EntropyCollapsePredictiveness: korreliert Entropiereduktion mit Erfolg?
- HiddenSingleUtility: Anteil Hidden Singles, die korrekte Entscheidungen erzeugen.
- BoundaryContractViolationRate: verletzte Boundary Contracts pro Run.

11.4 SelfModel-Metriken

- SelfDriftBoundAdherence: Anteil Runs innerhalb Drift Bound.
- DualTriggerPrecision: Anteil Trigger, die spaeter nutzbare Candidates erzeugen.
- CalibrationAcceptanceQuality: Anteil akzeptierter Kalibrierungen mit nachweislichem Nutzen.
- CarrierMigrationBenefit: Nutzen erlaubter Migrationen gegen Hold/Abort.

12 Agenten- und Code-Benchmarks

12.1 Taskklassen

Klasse	Beschreibung
SmallPatch	Einfache Codeaenderung mit deterministischen Tests.
RefactorWithConstraints	Refactoring mit Erhalt von API, Verhalten und Tests.
SpecToModule	Spezifikation in neues Modul transformieren.
BugLocalization	Fehler lokalisieren und minimalen Patch erzeugen.
MultiFileFeature	Feature ueber mehrere Dateien mit Integrationstests.
DocToSpec	Dokumentenrohstoff in normative Implementierungsspezifikation ueberfuehren.
RepoAudit	Architektur-, Sicherheits- oder Determinismusaudit.

12.2 Code-Agent-Metriken

- TestsPassedRatio
- CompileSuccess
- PatchMinimality
- RegressionRate
- SpecCoverage
- IterationCount
- TokenOrRuntimeCost
- HumanRepairMinutes
- ReplayableDecisionTrace

13 Failure Taxonomie

Failure	Definition	Pflichtreaktion
ReplayMismatch	Hash oder Bytes unterscheiden sich beim Replay.	Run invalidieren.
FalseCrystal	Crystal/Event ohne Ground-Truth-Entsprechung.	Sicherheitsmetrik erhoe-
MissedEvent	Ground-Truth-Event nicht er-	hen. Recall senken.
FalseHandoff	kannt. CognitionCandidateBundle fuehrt zu falscher Finalisierung.	Handoff precision senken.
OverHold	System blockiert korrekten Kan-	Hold calibration pruefen.
UnderHold	didaten. System laesst instabilen Kandida-	ten passieren.
MemoryMislead	Memory Hit verschlechtert Ergeb-	nis. FalseResonanceRate erhoe-
WormholeAbuse	Shortcut ohne Reason/Bud-	get/TTL oder schlechter Nutzen. Governance failure.
CalibrationLeakage	Testsplit beeinflusst Kalibrierung.	Evaluation invalidieren.

14 Reports

14.1 EvaluationSummaryReport

```
pub struct EvaluationSummaryReport {
  pub report_id: Hash256,
  pub evaluation_spec_hash: Hash256,
  pub generated_from_ledger: Hash256,
  pub variant_summaries: Vec<VariantSummary>,
  pub workload_summaries: Vec<WorkloadSummary>,
}
```



```

pub ablation_summaries: Vec<AblationSummary>,
pub capability_profiles: Vec<CapabilityProfile>,
pub statistical_summaries: Vec<StatisticalSummary>,
pub invalid_runs: Vec<Hash256>,
pub conclusion_flags: Vec<ConclusionFlag>,
}

```

14.2 CapabilityProfile

```

pub struct CapabilityProfile {
pub variant_id: String,
pub task_utility: CanonicalNumber,
pub replay_utility: CanonicalNumber,
pub safety_utility: CanonicalNumber,
pub cognition_utility: CanonicalNumber,
pub efficiency_utility: CanonicalNumber,
pub calibration_utility: CanonicalNumber,
pub robustness_utility: CanonicalNumber,
pub safety_adjusted_utility: CanonicalNumber,
}

```

14.3 AblationSummary

```

pub struct AblationSummary {
pub base_variant: String,
pub ablated_variant: String,
pub metric_deltas: Vec<MetricDelta>,
pub layer_marginal_utility: CanonicalNumber,
pub safety_regression: bool,
pub conclusion: AblationConclusion,
}

```

15 CLI-Schnittstelle

15.1 Subcommands

```

pse-eval-matrix init --template agent-cognition --out eval_spec.json
pse-eval-matrix validate --spec eval_spec.json
pse-eval-matrix plan --spec eval_spec.json --out eval_plan.json
pse-eval-matrix run --plan eval_plan.json --out runs/
pse-eval-matrix replay --ledger runs/ledger.json
pse-eval-matrix score --ledger runs/ledger.json --out summary.json
pse-eval-matrix ablate --spec eval_spec.json --matrix default --out ablations.json
pse-eval-matrix compare --summary summary.json --baseline B0_Baseline
pse-eval-matrix report --summary summary.json --format md,pdf,json
pse-eval-matrix export-fixtures --spec eval_spec.json --out fixtures/

```

15.2 CLI-Anforderungen

- **validate MUST** Schema, missing labels, metric declarations und split leakage pruefen.
- **plan MUST** eine deterministische Run-Liste erzeugen.
- **run MUST** RunLedger append-only schreiben.
- **replay MUST** alle TrialReports byte-identisch validieren.
- **score MUST** ausschliesslich aus Ledger und MetricSpecs rechnen.
- **ablate MUST** Variant Ladder und Feature Masks deterministisch erzeugen.
- **report MAY** PDF/Markdown/JSON exportieren, darf aber keine Metriken neu berechnen.

16 Feature Flags und Modulzuschnitt

16.1 Rust-Modulstruktur

```
crates/pse-eval-matrix/src/
  mod.rs
  spec.rs
  variants.rs
  workloads.rs
  datasets.rs
  ground_truth.rs
  metrics.rs
  scoring.rs
  calibration.rs
  runner.rs
  ledger.rs
  replay.rs
  ablation.rs
  statistics.rs
  reports.rs
  agent_tasks.rs
  cognition_metrics.rs
  failure_taxonomy.rs
  export.rs

tools/pse-eval-matrix-cli/src/main.rs
```

16.2 Feature Flags

Feature	Bedeutung
eval-matrix	Kernschemas, Metriken, Ledger, Scoring.
eval-cli	CLI-Tool.
eval-agent	Agenten- und Code-Workloads.
eval-cognition	Cognition-spezifische Metriken.
eval-streams	Streaming/Event-Detection-Workloads.

Feature	Bedeutung
eval-statistics	Bootstrap, Intervalle, Signifikanzberichte.
eval-reports	Markdown/PDF/HTML-Export.

17 Determinismus und Audit

17.1 Kanonisierung

1. Alle keyed collections **MUST** als `BTreeMap` oder `BTreeSet` serialisiert werden.
2. Alle Listen ohne semantische Ordnung **MUST** vor Hashing sortiert werden.
3. Alle Reports **MUST** JCS-kanonisch serialisiert werden.
4. Keine wall-clock timestamps im Audit-Pfad.
5. Keine nichtdeterministische RNG-Nutzung ohne `RunDescriptor-seed` und gespeicherte `Sampling-Policy`.
6. Plattform-Floats **MUST NOT** in Gate-, Metric- oder Score-Hashes eingehen.

17.2 Audit-Artifacts

Jede Evaluation **MUST** mindestens folgende Artefakte erzeugen:

- `EvaluationSpec`
- `EvaluationPlan`
- `DatasetManifest`
- `GroundTruthProfile`
- `EvaluationRunLedger`
- `TrialReport[]`
- `EvaluationSummaryReport`
- `ReplayVerificationReport`
- `AblationReport`

18 Benchmark-Matrix-Presets

18.1 Preset: Agent Cognition

- Varianten: B0, B2, B5, B6, B7.
- Workloads: `CodeAgentPatch`, `DocSynthesis`, `TraversalPuzzle`, `MemoryReuse`.
- Primaere Metriken: `TaskSuccess`, `RegressionRate`, `ManualIntervention`, `HandoffPrecision`, `ReplayIdentity`.
- Sekundäre Metriken: `PanoramaCoverage`, `MemoryHitUtility`, `ConstraintMass`, `Attractor-RankingAccuracy`.

18.2 Preset: Streaming Event Detection

- Varianten: B0 classical detector, B1 PSE Core, B7 full calibrated.
- Workloads: StreamEvent, AnomalyRegime.
- Primaere Metriken: Precision, Recall, F1, FalseCrystalRate, DetectionDelay.
- Sekundäre Metriken: GateCalibration, ThroughputObsPerSec, CrystalStability.

18.3 Preset: Post-Symbolic Ablation

- Varianten: vollständige Ladder B0–B7.
- Workloads: alle Pflichtfamilien mit minimalen Fixtures.
- Ziel: Layer Marginal Utility und Safety Regression bestimmen.

19 Qualitative Evaluation

19.1 Warum qualitative Evaluation erforderlich ist

Nicht alle relevanten Vorteile sind sofort als Zahl messbar. Spezifikationsarbeit, Dokumentenextraktion und Agentensteuerung benoetigen qualitative Bewertungen. Diese Bewertungen **MUST** strukturiert, wiederholbar und evidenzgebunden sein.

19.2 Rubrik

Dimension	Skala	Definition
StructuralFidelity	0–5	Wurden die relevanten Strukturprimitive korrekt extrahiert?
NeutralizationQuality	0–5	Wurde domänenspezifische Semantik korrekt entfernt?
ImplementationUsefulness	0–5	Kann ein Coding-Agent daraus direkt arbeiten?
Completeness	0–5	Sind Datentypen, Gates, Reports, Tests und DoD vorhanden?
NonRedundancy	0–5	Wurden Dopplungen vermieden?
FailureClarity	0–5	Sind Fehlerfaelle maschinenlesbar und handhabbar?

19.3 Review-Protokoll

Jedes qualitative Urteil **MUST** einen ReviewerReport erzeugen:

```
pub struct ReviewerReport {
  pub reviewer_id_hash: Hash256,
  pub artifact_hash: Hash256,
  pub rubric_scores: BTreeMap<String, u8>,
}
```

```
pub blocking_issues: Vec<String>,
pub evidence_refs: Vec<EvidenceRef>,
}
```

20 Definition of Done

Eine Implementierung von PSE-EVAL-MATRIX-01 gilt als abgeschlossen, wenn:

1. alle oeffentlichen Typen dokumentiert sind;
2. `EvaluationSpec` validierbar und content-addressed ist;
3. `EvaluationPlan` deterministisch aus `EvaluationSpec` erzeugt wird;
4. `RunLedger` append-only und hash-chained ist;
5. `TrialReports` JCS-kanonisch und replayfaehig sind;
6. mindestens drei Presets implementiert sind: Agent Cognition, Streaming Event Detection, Post-Symbolic Ablation;
7. Ablation Ladder B0–B7 erzeugbar ist;
8. Replay-Verletzungen Runs invalidieren;
9. Scoring ausschliesslich aus Reports und MetricSpecs erfolgt;
10. `StatisticalSummary` Confidence Intervals erzeugen kann;
11. CLI `init|validate|plan|run|replay|score|ablate|compare|report` funktioniert;
12. Golden Fixtures fuer mindestens zwei Workload-Familien enthalten sind;
13. Tests fuer Determinismus, Replay, Metric Direction, Invalid Runs, Ablation Deltas und Score-Stabilitaet bestehen;
14. keine externe Netzwerkabhaengigkeit im Kern besteht;
15. `cargo test -workspace` besteht.

21 Coding-Agent-Auftrag

PATCH-ID: PSE-EVAL-MATRIX-01

TITLE: Empirical Benchmark Matrix for Post-Symbolic Cognition Systems

GOAL:

Implementiere eine universell nutzbare Evaluationsmatrix fuer PSE, pse-traverse, horizon, cognition und verwandte post-symbolische Kognitionssysteme. Das Ergebnis darf kein simples Benchmark-Harness sein, sondern muss Systemvarianten, Workloads, Domänen, Metriken, Ablationen, Replay, Statistik, qualitative Rubriken und Reports verbinden.

MUST:

- crate crates/pse-eval-matrix anlegen;
- CLI tools/pse-eval-matrix-cli anlegen;
- `EvaluationSpec`, `DatasetManifest`, `GroundTruthProfile`, `SystemVariantSpec`, `WorkloadSpec`, `MetricSpec`, `EvaluationRunLedger`, `TrialReport`, `EvaluationSummaryReport` implementieren;
- Variant Ladder B0_Baseline bis B7_FullStack unterstuetzen;

- Agent Cognition, Streaming Event Detection und Post-Symbolic Ablation Presets implementieren;
- Ablation-Plan deterministisch erzeugen;
- RunLedger append-only und hash-chained implementieren;
- JCS-canonical serialization fuer Reports verwenden;
- Replay-Verifikation implementieren;
- Scoring und CapabilityProfile implementieren;
- qualitative ReviewerRubric implementieren;
- CLI init/validate/plan/run/replay/score/ablate/compare/report bereitstellen;
- Golden Fixtures und Tests genaess DoD liefern.

MUST NOT:

- Metriken nachtraeglich ohne Spec-Aenderung umdeuten;
- Testdaten fuer Kalibrierung leaken;
- Runs mit ReplayMismatch als gueltig zaehlen;
- nichtdeterministische RNG- oder wall-clock-Daten in Audit-Hashes schreiben;
- Plattform-Floats in Score-/Metric-Hashes verwenden.

SHOULD:

- bestehende pse-traverse Reports als Inputs akzeptieren;
- PSE core benchmarks und cognition CLI outputs integrieren;
- paired comparisons und Bootstrap-CI bereitstellen;
- Markdown, JSON und optional PDF Summary Reports exportieren;
- alle Feature Flags klar trennen.

22 Referenz-Zustandsmaschine

Von	Nach	Bedingung
Idle	SpecLoaded	EvaluationSpec vorhanden.
SpecLoaded	Validated	Schema, Labels, Splits, Metrics gueltig.
Validated	Planned	Deterministischer EvaluationPlan erzeugt.
Planned	Running	RunQueue initialisiert.
Running	TrialRecorded	TrialReport erzeugt und ghasht.
TrialRecorded	LedgerAppended	RunLedger append-only aktualisiert.
LedgerAppended	Replayed	ReplayHash verifiziert.
Replayed	Scored	MetricSpecs angewendet.
Scored	Ablated	AblationSummaries berechnet.
Ablated	Reported	EvaluationSummaryReport erzeugt.
Any	Invalid	ReplayMismatch, schema break, leakage, missing primary metric.

23 Schlussformel

Ein post-symbolisches Kognitionssystem gilt innerhalb dieser Matrix nur dann als empirisch verbessert, wenn:

$$\Delta U_{task} > 0 \wedge \Delta U_{safety} \geq 0 \wedge ReplayIdentity = 1 \wedge InvalidRunRate \leq \epsilon \wedge LMU_{target} > 0.$$

Ist eine dieser Bedingungen nicht erfuehlt, lautet das Ergebnis nicht *Verbesserung*, sondern *diagnostischer Befund*.